

MAPA DE FUNÇÕES, CÁLCULOS E FÓRMULAS MATEMÁTICAS

Referência Técnica Exaustiva do Código-Fonte — Computação Gráfica & Processamento Digital de Imagens

1. PROJETO 1 — QUESTÃO 1: RASTERIZAÇÃO, CÔNICAS E SPLINES BÉZIER

■ Função: `reta_dda(x0, y0, x1, y1, cor)`

■ Arquivo & Linha: `Projeto 1/Projeto_C6_Final/mod_primitivas/Questao1/core/cg_utils.py` -> L180

■ Cálculo / Fórmula: $dx = x1 - x0$, $dy = y1 - y0$; $passos = \max(|dx|, |dy|)$; $inc_x = dx/passos$; $inc_y = dy/passos$; $x += inc_x$, $y += inc_y$

■ O que o código faz: Calcula o coeficiente angular m e realiza incrementos contínuos em ponto flutuante (float) a cada iteração, arredondando $round(x)$, $round(y)$ para acender o pixel discreto.

■ Função: `reta_ponto_medio(x0, y0, x1, y1, cor)`

■ Arquivo & Linha: `Projeto 1/Projeto_C6_Final/mod_primitivas/Questao1/core/cg_utils.py` -> L198

■ Cálculo / Fórmula: $d_inicial = 2*dy - dx$; se $d < 0$: $d += 2*dy$ (escolhe E); se $d \geq 0$: $y++$, $d += 2*(dy - dx)$ (escolhe NE)

■ O que o código faz: Algoritmo de Bresenham inteiro para retas. Avalia o sinal da função implícita $F(x, y) = 2*dy*x - 2*dx*y + c$ no ponto médio entre os pixels candidatos. Trata os 8 oitantes espelhando dx , dy .

■ Função: `circ_ponto_medio(xc, yc, r, cor)`

■ Arquivo & Linha: `Projeto 1/Projeto_C6_Final/mod_primitivas/Questao1/core/cg_utils.py` -> L264

■ Cálculo / Fórmula: $d_inicial = 1 - r$; se $d < 0$: $d += 2*x + 3$; se $d \geq 0$: $y--$, $d += 2*(x - y) + 5$; Simetria de 8 oitantes: $(\pm x, \pm y)$, $(\pm y, \pm x)$

■ O que o código faz: Bresenham para circunferências. Avalia $F(x+1, y-1/2) = (x+1)^2 + (y-1/2)^2 - r^2$. Inicia em $(0, r)$ até $x = y$ e plota 8 pixels simultâneos por simetria axial e diagonal.

■ Função: `elipse_ponto_medio(xc, yc, rx, ry, cor)`

■ Arquivo & Linha: `Projeto 1/Projeto_C6_Final/mod_primitivas/Questao1/core/cg_utils.py` -> L285

■ Cálculo / Fórmula: Região 1 ($dy/dx > -1$): $d1 = ry^2 - rx^2*ry + 0.25*rx^2$; Região 2 ($dy/dx < -1$): $d2 = ry^2*(x+0.5)^2 + rx^2*(y-1)^2 - rx^2*ry^2$

■ O que o código faz: Algoritmo de Ponto Médio para Elipses. Na Região 1, avança $x++$ e escolhe y ou $y-1$ enquanto $2*ry^2*x \leq 2*rx^2*y$. Na Região 2, avança $y--$ e escolhe x ou $x+1$. Plota 4 quadrantes por simetria.

■ Função: `parabola(xc, yc, a, cor)` & `hiperbole(xc, yc, a, b, cor)`

■ Arquivo & Linha: `Projeto 1/Projeto_C6_Final/mod_primitivas/Questao1/core/cg_utils.py` -> L326, L338

■ Cálculo / Fórmula: Parábola: $y = a(x - xc)^2 + yc$; Hipérbole: $y = \pm(b/a)*\sqrt{(x - xc)^2 - a^2}$

■ O que o código faz: Rasterização de cônicas por varredura explícita com passo adaptativo em x e plotagem dos ramos simétricos superior e inferior.

■ Função: `spline_bezier_cubica(p0, p1, p2, p3, passos, cor)`

■ Arquivo & Linha: `Projeto 1/Projeto_C6_Final/mod_primitivas/Questao1/core/cg_utils.py` -> L355

■ Cálculo / Fórmula: $P(t) = (1-t)^3 \cdot P_0 + 3(1-t)^2 \cdot t \cdot P_1 + 3(1-t) \cdot t^2 \cdot P_2 + t^3 \cdot P_3$, para $t \in [0, 1]$

■ O que o código faz: Curva de Bézier cúbica com 4 pontos de controle via Polinômios de Bernstein de grau 3. Conecta pontos sucessivos com a reta de ponto médio (Bresenham) discreta.

■ Função: `T(tx, ty)`, `R(theta)`, `S(sx, sy)`, `Sh(shx, shy)`

■ Arquivo & Linha: `Projeto 1/Projeto_C6_Final/mod_primitivas/Questao1/core/cg_utils.py` -> L375-L415

■ Cálculo / Fórmula: $T = [[1, 0, tx], [0, 1, ty], [0, 0, 1]]$; $R = [[\cos, -\sin, 0], [\sin, \cos, 0], [0, 0, 1]]$; $S = [[sx, 0, 0], [0, sy, 0], [0, 0, 1]]$; $Sh = [[1, shx, 0], [shy, 1, 0], [0, 0, 1]]$

■ O que o código faz: Construtores de matrizes de transformação 2D em coordenadas homogêneas 3x3. Permitem aplicar translações, rotações, escalas e cisalhamentos por multiplicação matricial linear.

■ Função: `_dlg_rotacionar()` & `_dlg_composicao()`

■ **Arquivo & Linha:** [Projeto 1/Projeto_CG_Final/mod_primitivas/Questao1/apps/transf2d_ui.py](#) -> L435, L478

■ **Cálculo / Fórmula:** Pivô no Centro: $M = T(x_c, y_c) \cdot R(\theta) \cdot T(-x_c, -y_c)$; Composição Sequencial: $M_{total} = M_n \cdot \dots \cdot M_2 \cdot M_1$

■ **O que o código faz:** Calcula o baricentro $(x_c, y_c) = (\sum x/N, \sum y/N)$ para rotacionar/escalar o polígono mantendo-o no lugar. A composição multiplica matrizes na ordem correta e aplica aos vértices $P' = M_{total} \cdot [x, y, 1]^T$.

2. PROJETO 1 — QUESTÕES 2, 3 E 4: RECORTES 2D, MODELAGEM 3D E IMAGENS PGM

■ **Função:** `cohen_sutherland_clip(x0, y0, x1, y1, xmin, ymin, xmax, ymax)`

■ **Arquivo & Linha:** `Projeto 1/Projeto_C6_Final/mod_recortes/Questao2/apps/cohen_sutherland_ui.py` -> L70

■ **Cálculo / Fórmula:** Outcodes [TOP=8, BOTTOM=4, RIGHT=2, LEFT=1]; Se $c_0|c_1 == 0$: Aceite; Se $c_0 \& c_1 != 0$: Rejeição; Interseção: $x = x_0 + (y_{borda} - y_0)/m$ ou $y = y_0 + m*(x_{borda} - x_0)$

■ **O que o código faz:** Algoritmo de recorte de retas por códigos de região de 4 bits. Calcula a interseção analítica da reta com a borda externa ativa até que ambos os pontos estejam dentro da janela ou descartados.

■ **Função:** `sutherland_hodgman_clip(poligono, bordas)`

■ **Arquivo & Linha:** `Projeto 1/Projeto_C6_Final/mod_recortes/Questao2/apps/sutherland_hodgman_ui.py` -> L85

■ **Cálculo / Fórmula:** Para cada borda (Esq, Dir, Abaixo, Acima): 1. In->In: add V2; 2. In->Out: add I; 3. Out->Out: descarta; 4. Out->In: add I, add V2

■ **O que o código faz:** Algoritmo de recorte de polígonos convexos e côncavos. Processa a lista de vértices contra cada uma das 4 retas de contorno da janela, gerando o polígono final recortado.

■ **Função:** `translate3d(), scale3d(), rotate_x(), rotate_y(), rotate_z()`

■ **Arquivo & Linha:** `Projeto 1/Projeto_C6_Final/mod_3d/Questao3/core/cg_utils.py` -> L75-L95

■ **Cálculo / Fórmula:** $R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c & -s & 0 \\ 0 & s & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$; $R_y = \begin{bmatrix} c & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ -s & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$; $R_z = \begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

■ **O que o código faz:** Matrizes 4x4 em coordenadas homogêneas para transformações espaciais tridimensionais aplicadas a vértices $[x, y, z, 1]^T$.

■ **Função:** `projection_isometric() & map_window_to_viewport_rect()`

■ **Arquivo & Linha:** `Projeto 1/Projeto_C6_Final/mod_3d/Questao3/Questao3.py` -> L35, L60

■ **Cálculo / Fórmula:** $M_{proj} = R_x(35.264^\circ) \cdot R_y(45^\circ)$; $x_v = V_{xmin} + (x_w - W_{xmin}) \cdot (V_w/W_w)$; $y_v = V_{ymin} + (W_{ymax} - y_w) \cdot (V_h/W_h)$

■ **O que o código faz:** Projeta os vértices 3D em 2D na Projeção Paralela Isométrica (ângulos iguais entre os eixos) e mapeia linearmente da Janela do Mundo para as coordenadas físicas da Viewport.

■ **Função:** `aplicar_transformacao_afim_inversa(imagem, matriz_afim)`

■ **Arquivo & Linha:** `Projeto 1/Projeto_C6_Final/mod_imagens/Questao4/transformacoes.py` -> L120

■ **Cálculo / Fórmula:** Para cada (x_{out}, y_{out}) na imagem de saída: $[x_{in}, y_{in}, 1]^T = M^{-1} \cdot [x_{out}, y_{out}, 1]^T$; $I_{out}(x_{out}, y_{out}) = I_{in}(\text{round}(x_{in}), \text{round}(y_{in}))$

■ **O que o código faz:** Mapeamento Inverso para transformações afins (Rotação, Escala, Cisalhamento) em imagens PGM. Garante que todos os pixels da imagem destino recebam valor, eliminando furos (gaps).

3. PROJETO 2 — PROCESSAMENTO DIGITAL DE IMAGENS: CORE & OPERAÇÕES

■ Função: `carregar_imagem(caminho)` & `salvar_imagem(imagem, caminho)`

■ Arquivo & Linha: `Projeto 2/PI/src/laboratorio_imagens/core/io_netpbm.py` -> L88, L180

■ Cálculo / Fórmula: **Parser NetPBM: P1 (Binário ASCII 0/1), P2 (Escala de cinza texto ASCII 0..255), P5 (Escala de cinza binária em bytes brutos)**

■ O que o código faz: Lê os tokens do cabeçalho NetPBM (Mágico, Largura, Altura, MaxVal) e armazena os dados em matriz NumPy 2D uint8. Rejeita arquivos fora do padrão estrito.

■ Função: `aplicar_correlacao(matriz, mascara)` & `aplicar_mediana(matriz, tamanho=3)`

■ Arquivo & Linha: `Projeto 2/PI/src/laboratorio_imagens/core/filtros_espaciais.py` -> L25, L45

■ Cálculo / Fórmula: **Convolução 2D: $g(x, y) = \sum \sum w(s, t) \cdot f(x+s, y+t)$; Mediana: $g(x, y) = \text{median}\{f(x+i, y+j) \mid i, j \in [-1, 1]\}$**

■ O que o código faz: Convolução linear 2D com reflexão de bordas. O filtro da mediana ordena os 9 vizinhos e seleciona o valor central (50º percentil), eliminando ruído impulsivo sem degradar bordas.

■ Função: `operador_roberts()`, `operador_prewitt()`, `operador_sobel()`

■ Arquivo & Linha: `Projeto 2/PI/src/laboratorio_imagens/core/filtros_espaciais.py` -> L264-L318

■ Cálculo / Fórmula: **Magnitude do Gradiente: $G = |G_x| + |G_y|$ ou $\sqrt{G_x^2 + G_y^2}$; $G_x = \text{Convolução}(I, M_x)$; $G_y = \text{Convolução}(I, M_y)$**

■ O que o código faz: Calcula derivadas direcionais de 1ª ordem. Roberts usa diferenças diagonais 2x2; Prewitt usa diferenças de médias 3x3; Sobel usa ponderação 2 no centro para suavizar ruído.

■ Função: `filtro_high_boost(matriz, fator_a=1.2)`

■ Arquivo & Linha: `Projeto 2/PI/src/laboratorio_imagens/core/filtros_espaciais.py` -> L325

■ Cálculo / Fórmula: **HighBoost = $A \cdot \text{Imagem_Original} - \text{Passa_Baixas} = (A - 1) \cdot \text{Original} + \text{Passa_Altas}$**

■ O que o código faz: Filtro de realce de alta frequência com fator de amplificação $A \geq 1$. Para $A = 1$, equivale à máscara de nitidez clássica (Unsharp Masking); para $A > 1$, reforça os detalhes sobre a imagem base.

■ Função: `somar_imagens()`, `subtrair_imagens()`, `multiplicar_imagens()`, `dividir_imagens()`

■ Arquivo & Linha: `Projeto 2/PI/src/laboratorio_imagens/core/operacoes_aritmeticas.py` -> L20-L80

■ Cálculo / Fórmula: **Truncamento: $\text{clip}(A \pm B, 0, 255)$; Normalização: $((I - \min) / (\max - \min)) \cdot 255$; Multiplicação: $(A \cdot B) / 255$**

■ O que o código faz: Operações aritméticas ponto a ponto entre matrizes de imagem com tratamento explícito de overflow (>255) e underflow (<0) via corte estrito ou remapeamento linear de contraste.

■ Função: `negativo()`, `transformacao_gamma()`, `transformacao_logaritmica()`, `funcao_sigmoide()`

■ Arquivo & Linha: `Projeto 2/PI/src/laboratorio_imagens/core/transformacoes_intensidade.py` -> L15-L75

■ Cálculo / Fórmula: **Negativo: $s = 255 - r$; Gamma: $s = c \cdot 255 \cdot (r/255)^\gamma$; Log: $s = c \cdot (255/\ln(256)) \cdot \ln(1 + r)$; Sigmóide: $s = 255 / (1 + e^{-k(r - x_0)})$**

■ O que o código faz: Transformações pontuais de intensidade não-lineares. Gamma ajusta o contraste de tons escuros/claros; Log expande valores escuros comprimindo claros; Sigmóide cria transição suave em S.

■ Função: `equalizar_histograma(matriz)`

■ Arquivo & Linha: `Projeto 2/PI/src/laboratorio_imagens/core/histograma.py` -> L40

■ Cálculo / Fórmula: **$p(r_k) = n_k / N$; $\text{CDF}(k) = \sum_{j=0}^k p(r_j)$ de $j=0$ até k ; $s_k = \text{round}(255 \cdot \text{CDF}(k))$**

■ O que o código faz: Equalização de histograma via Função de Distribuição Acumulada (CDF). Mapeia os níveis de cinza para distribuição uniforme, maximizando o contraste global da imagem.

4. PROJETO 2 — MORFOLOGIA MATEMÁTICA E MORFISMO TEMPORAL (DELAUNAY)

■ Função: erosao(matriz, kernel) & dilatacao(matriz, kernel)

■ Arquivo & Linha: [Projeto 2/PI/src/laboratorio_imagens/core/morfologia.py](#) -> L25, L45

■ Cálculo / Fórmula: Erosão: $(A \ominus B)(x, y) = \min_{\{i,j \in B\}} \{A(x+i, y+j)\}$; Dilatação: $(A \oplus B)(x, y) = \max_{\{i,j \in B\}} \{A(x-i, y-j)\}$

■ O que o código faz: Operadores morfológicos fundamentais com elemento estruturante B (kernel 3x3 cruz/quadrado). Em imagens em tons de cinza, a erosão calcula o mínimo local e a dilatação calcula o máximo local.

■ Função: abertura(), fechamento(), gradiente_morfologico(), top_hat(), bottom_hat()

■ Arquivo & Linha: [Projeto 2/PI/src/laboratorio_imagens/core/morfologia.py](#) -> L65-L125

■ Cálculo / Fórmula: Abertura: $(A \ominus B) \oplus B$; Fechamento: $(A \oplus B) \ominus B$; Gradiente: $(A \oplus B) - (A \ominus B)$; TopHat: $A - (A \ominus B)$; BottomHat: $(A \oplus B) - A$

■ O que o código faz: Abertura remove ruídos claros menores que B; Fechamento fecha orifícios escuros; Gradiente destaca contornos morfológicos; Top-Hat extrai elementos brilhantes isolados; Bottom-Hat extrai vales escuros.

■ Função: hit_or_miss(matriz, b1, b2)

■ Arquivo & Linha: [Projeto 2/PI/src/laboratorio_imagens/core/morfologia.py](#) -> L135

■ Cálculo / Fórmula: Hit-or-Miss: $(A \ominus B1) \cap (A^c \ominus B2)$

■ O que o código faz: Transformação acerto-ou-erro para detecção de configurações geométricas e padrões específicos. Realiza a interseção entre a erosão do objeto por B1 e a erosão do fundo complementar por B2.

■ Função: preparar_morfismo(matriz_ini, matriz_fin, pts_ini, pts_fin)

■ Arquivo & Linha: [Projeto 2/PI/src/laboratorio_imagens/core/morfismo.py](#) -> L80

■ Cálculo / Fórmula: Triangulação de Delaunay sobre os pontos médios: $P_{med} = 0.5 \cdot P_{ini} + 0.5 \cdot P_{fin}$; gera lista de triângulos $T = [(i0, i1, i2)]$

■ O que o código faz: Gera a malha triangular ótima que cobre o domínio de ambas as imagens, evitando triângulos muito finos e garantindo correspondência topológica 1:1 entre os marcos anatômicos faciais.

■ Função: gerar_frame_preparado(preparacao, t)

■ Arquivo & Linha: [Projeto 2/PI/src/laboratorio_imagens/core/morfismo.py](#) -> L140

■ Cálculo / Fórmula: 1. Vértices interpolados: $P(t) = (1-t) \cdot P_{ini} + t \cdot P_{fin}$; 2. Deformação afim por Coordenadas Baricêntricas; 3. Cross-Dissolve: $I(t) = (1-t) \cdot I_{def_ini} + t \cdot I_{def_fin}$

■ O que o código faz: Para cada instante de tempo $t \in [0, 1]$, calcula a deformação afim triangular através de coordenadas baricêntricas (α, β, γ com $\alpha + \beta + \gamma = 1$) e combina as intensidades luminosas.